

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3.1

Группа: Р3407

Выполнил:

Батманов Д.Е.

Проверил:

к.т.н. преподаватель

Белозубов А.В.

Санкт-Петербург

2026г.

Оглавление	
<u>ВВЕДЕНИЕ</u>	3
<u>ВЫПОЛНЕНИЕ</u>	6
<u>ЗАКЛЮЧЕНИЕ</u>	21

ВВЕДЕНИЕ

Контейнеризация представляет собой метод виртуализации на уровне операционной системы, при котором приложения запускаются в изолированных пользовательских пространствах, называемых контейнерами. В отличие от традиционной виртуализации, контейнеры используют общее ядро операционной системы хоста, что обеспечивает значительно меньшее потребление ресурсов и практически мгновенный запуск приложений.

Лабораторная работа охватывает полный цикл работы с платформой Docker: от установки и освоения базовых команд до создания многокомпонентных приложений с использованием Docker Compose и публикации проектов в облачном реестре DockerHub.

Работа выполняется в двухуровневой архитектуре виртуализации: физическая система Windows с гипервизором Oracle VirtualBox, внутри которого развернута гостевая операционная система Ubuntu Linux, служащая средой для работы с Docker и запуска контейнеризированных приложений.

Описание используемых инструментов:

Docker — платформа для автоматизации развертывания приложений в легковесных переносимых контейнерах, использующая технологии изоляции на уровне ядра Linux.

Dockerfile — текстовый файл с инструкциями для автоматизированной сборки образов Docker.

Docker Compose — инструмент для определения и запуска многоконтейнерных приложений через конфигурационные файлы формата YAML.

DockerHub — облачный реестр для хранения, распространения и версионирования образов Docker.

VirtualBox — система виртуализации от Oracle для создания изолированной Unix-среды на физической системе Windows.

Nginx/Apache2 — веб-серверы для обработки HTTP-запросов и обслуживания веб-приложений.

MariaDB — реляционная СУБД с открытым исходным кодом, полностью совместимая с MySQL.

NextCloud — платформа для развертывания частного облачного хранилища с функциями синхронизации файлов и совместной работы.

phpVirtualBox — веб-интерфейс для удаленного управления виртуальными машинами VirtualBox.

В процессе работы используются различные подходы к контейнеризации: однокомпонентные контейнеры с ручной конфигурацией, Dockerfile-based развертывание для создания кастомизированных образов, и оркестрация через Docker Compose для объединения взаимосвязанных контейнеров в единую инфраструктуру.

Практическая часть включает работу с persistent volumes для обеспечения сохранности данных, проброс портов для доступа к сервисам, настройку межконтейнерного взаимодействия, а также публикацию созданных образов в публичный реестр DockerHub.

Цель работы

Целью данной лабораторной работы является освоение технологий контейнеризации с использованием платформы Docker, изучение принципов создания, конфигурирования и оркестрации контейнеров, а также получение практических навыков работы с облачными реестрами образов и многокомпонентными приложениями.

Задачи работы

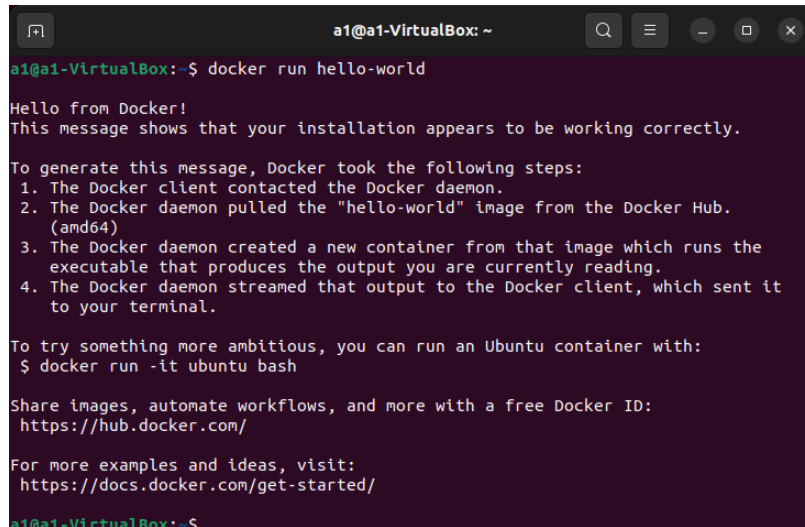
1. Настроить окружение виртуализации с установкой Docker и VirtualBox в Unix-системе.
2. Освоить базовые команды Docker для управления контейнерами и образами.
3. Создать пользовательский образ веб-сервера с использованием Dockerfile и персональной HTML-страницей.
4. Развернуть контейнер с СУБД MariaDB и выполнить создание базы данных через терминальный клиент.
5. Сконфигурировать и запустить платформу NextCloud с регистрацией пользователей.
6. Развернуть phpVirtualBox для веб-управления виртуальными машинами.

7. Создать многокомпонентное приложение с использованием Docker Compose.
8. Зарегистрироваться на платформе DockerHub и опубликовать созданный проект.
9. Освоить работу с готовыми образами из DockerHub.
10. Систематизировать и задокументировать основные команды Docker, использованные в работе.

ВЫПОЛНЕНИЕ

1. Установка Docker и VirtualBox

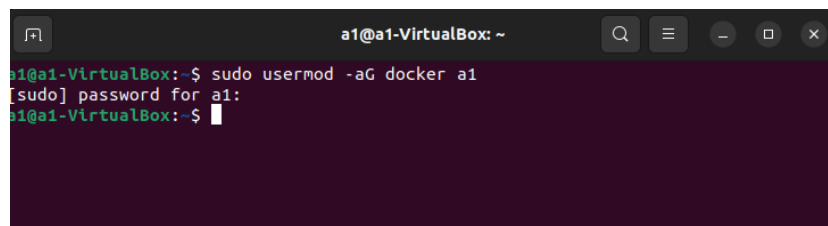
На Unix-систему (Ubuntu, запущенная на виртуальной машине) установили docker (Рисунок 1):



```
a1@a1-VirtualBox: ~  
a1@a1-VirtualBox:~$ docker run hello-world  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/  
a1@a1-VirtualBox:~$
```

Рисунок 1 – установленный docker

Также пользователь был добавлен в группу docker для запуска команд без sudo (Рисунок 2):



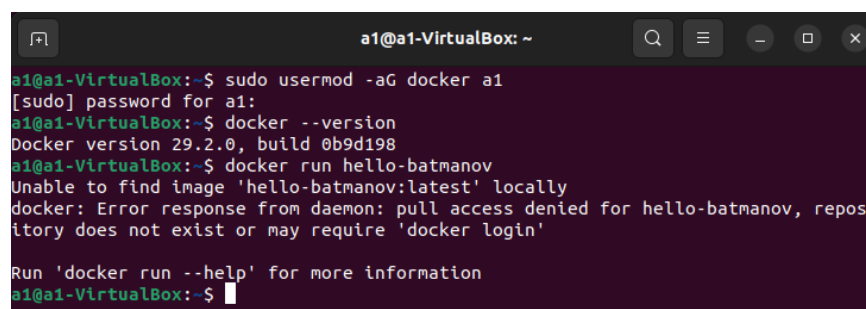
```
a1@a1-VirtualBox: ~  
a1@a1-VirtualBox:~$ sudo usermod -aG docker a1  
[sudo] password for a1:  
a1@a1-VirtualBox:~$
```

Рисунок 2 – настройка Докера

2. Запуск тестового контейнера

Выполняем команду

```
docker run hello-batmanov
```



```
a1@a1-VirtualBox: ~  
a1@a1-VirtualBox:~$ sudo usermod -aG docker a1  
[sudo] password for a1:  
a1@a1-VirtualBox:~$ docker --version  
Docker version 29.2.0, build 0b9d198  
a1@a1-VirtualBox:~$ docker run hello-batmanov  
Unable to find image 'hello-batmanov:latest' locally  
docker: Error response from daemon: pull access denied for hello-batmanov, repository does not exist or may require 'docker login'  
  
Run 'docker run --help' for more information  
a1@a1-VirtualBox:~$
```

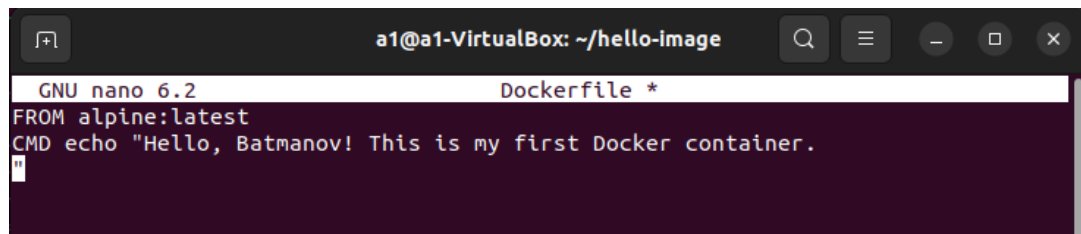
Рисунок 3 – ошибка при запуске

Команда выдает ошибку (Рисунок 3), т. к. указанного контейнера нет ни локально, ни в удаленном репозитории (что логично, ведь мы его еще не создали).

Для успешного запуска команды создаем контейнер. Сначала создаем директорию и Dockerfile внутри нее (Рисунок 4 и Рисунок 5):

```
a1@a1-VirtualBox:~$ cd hello-image
a1@a1-VirtualBox:~/hello-image$ nano Dockerfile
```

Рисунок 4 – создание dockerfile



The screenshot shows a terminal window titled "a1@a1-VirtualBox: ~/hello-image". Inside, the GNU nano 6.2 editor is open, editing a file named "Dockerfile *". The content of the Dockerfile is as follows:

```
FROM alpine:latest
CMD echo "Hello, Batmanov! This is my first Docker container."
```

Рисунок 5 – содержимое dockerfile

После чего собираем и запускаем созданный контейнер (Рисунок 6):

```
a1@a1-VirtualBox:~/hello-image$ docker run hello-batmanov:latest
Hello, Batmanov! This is my first Docker container.
```

Рисунок 6 – запуск контейнера

Как видим, контейнер успешно запустился, теперь команда работает.

3. Запуск контейнера с Nginx

Создаем необходимые разделы (Рисунок 7), после чего создаем HTML страницу, которая будет использоваться в контейнере (Рисунок 8).

```

a1@a1-VirtualBox:~$ mkdir -p ~/nginx-project
a1@a1-VirtualBox:~$ cd ~/nginx-project
a1@a1-VirtualBox:~/nginx-project$ mkdir html
a1@a1-VirtualBox:~/nginx-project$ sudo mkdir -p /home/www/html
[sudo] password for a1:
a1@a1-VirtualBox:~/nginx-project$ sudo chown -R $USER /home/www/html
a1@a1-VirtualBox:~/nginx-project$ cp ~/nginx-project/html/index.html /home/www/html/
cp: cannot stat '/home/a1/nginx-project/html/index.html': No such file or directory
a1@a1-VirtualBox:~/nginx-project$ sudo chown -R $USER:USER /home/www/html
chown: invalid group: 'a1:USER'
a1@a1-VirtualBox:~/nginx-project$ sudo chown -R $USER:$USER /home/www/html
a1@a1-VirtualBox:~/nginx-project$ cp ~/nginx-project/html/index.html /home/www/html/
cp: cannot stat '/home/a1/nginx-project/html/index.html': No such file or directory
a1@a1-VirtualBox:~/nginx-project$ ls
html
a1@a1-VirtualBox:~/nginx-project$ ls html
a1@a1-VirtualBox:~/nginx-project$ cd html
a1@a1-VirtualBox:~/nginx-project/html$ ls
a1@a1-VirtualBox:~/nginx-project/html$ vi index.html
a1@a1-VirtualBox:~/nginx-project/html$ cat index.html
<!DOCTYPE html>
<html>
<body>
Hello world! I'm Batmanov
</body>
</html>
a1@a1-VirtualBox:~/nginx-project/html$ cd ..
a1@a1-VirtualBox:~/nginx-project$ cp ~/nginx-project/html/index.html /home/www/html/
a1@a1-VirtualBox:~/nginx-project$

```

Рисунок 7 – создание разделов

```

a1@a1-VirtualBox:~/nginx-project/html$ cat index.html
<!DOCTYPE html>
<html>
<body>
Hello world! I'm Batmanov
</body>
</html>

```

Рисунок 8 – создание HTML страницы

Далее создаем Dockerfile с настройками контейнера (Рисунок 9):

```

a1@a1-VirtualBox:~/nginx-project$ cat Dockerfile
FROM nginx:latest

WORKDIR /usr/share/nginx/html

EXPOSE 8080

RUN sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf

```

Рисунок 9 – dockerfile для Nginx

Собираем и запускаем контейнер с пробросом портов (Рисунок 10):


```
alga1-VirtualBox: /nginx-project$ docker build -t nginx-batmanov .
[+] Building 33.5s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> == transferring dockerfile: 175B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> == transferring context: 2B
=> [1/3] FROM docker.io/library/nginx:latest@sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
=> == resolve docker.io/library/nginx:latest@sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
=> == sha256: eaf8753feae0b5aaadb86ac2cb972ff57cfff12f877ad50c548fde8092e85e7fb 1.40kB / 1.40kB
=> == sha256: 57f0dd1befe210f3a7186fe46ad1552d4bc8864701c8cec580676144bc37d425 1.21kB / 1.21kB
=> == sha256: 10b68cfefee1d8726f1f754e9f566b73bd4df2531476315e6ac55b4fe18e1717 404B / 404B
=> == sha256: 508799c304244b0545f8f1d54d86d3a17512c219cd0edd3d5f5b0f68f5c6f93 955B / 955B
=> == sha256: d989100b8a84afca8cb4b5bcc62beec741cb69e181fe7815d79e4aa04c36ca59 628B / 628B
=> == sha256: 700146c8ad64762607eb0076d9fe376bed85c65f27650a4f3b76fc66f72e9846 32.51MB / 33.07MB
=> == sha256: 119d43eec815e5f9a47da3a7d59454581b1e204b0c34db86f171b7ceb3336533 29.77MB / 29.77MB
=> == extracting sha256: 119d43eec815e5f9a47da3a7d59454581b1e204b0c34db86f171b7ceb3336533
=> == extracting sha256: 700146c8ad64762607eb0076d9fe376bed85c65f27650a4f3b76fc66f72e9846
=> == extracting sha256: d989100b8a84afca8cb4b5bcc62beec741cb69e181fe7815d79e4aa04c36ca59
=> == extracting sha256: 508799c304244b0545f8f1d54d86d3a17512c219cd0edd3d5f5b0f68f5c6f93
=> == extracting sha256: 10b68cfefee1d8726f1f754e9f566b73bd4df2531476315e6ac55b4fe18e1717
=> == extracting sha256: 57f0dd1befe210f3a7186fe46ad1552d4bc8864701c8cec580676144bc37d425
=> == sha256: eaf8753feae0b5aaadb86ac2cb972ff57cfff12f877ad50c548fde8092e85e7fb
=> [2/3] WORKDIR /usr/share/nginx/html
=> [3/3] RUN sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf
=> == exporting image
=> == exporting layers
=> == exporting manifest sha256: 7edbf8d3e9fff899578de9c2977b2235506c7305eb93f1d7ce00e0a689ff4451
=> == exporting config sha256: 7c208459eb67e5616d7627df2b27201b01dbd91f2da1db6a527097ebceccf0fe
=> == exporting attestation manifest sha256: ee8015fcd4dc87298e0b077ea44e89e728b2636b2a5b0df38053d30572bc6415
=> == exporting manifest list sha256: 1ebdfb70ea596999629d2ed255be2cc3a744571b07203035af4e58c1333cac65
=> == naming to docker.io/library/nginx-batmanov:latest
=> == unpacking to docker.io/library/nginx-batmanov:latest
alga1-VirtualBox: /nginx-project$ docker run -d -p 8080:8080 -v /home/www/html:/usr/share/nginx/html --name web-batmanov nginx-batmanov
9807e7399b6c56b6a1bcb110bcadb041904f820817fcede3e0969ee99de923ee
```

Рисунок 10 – сборка и запуск контейнера

Где:

-d – запуск в фоне

-p 8080:8080 – проброс порта

-v /home/www/html:/usr/share/nginx/html – монтирование директории хоста

--name web-batmanov – имя контейнера

Открываем локалхост в Firefox и видим, что все работает, страница отображается:

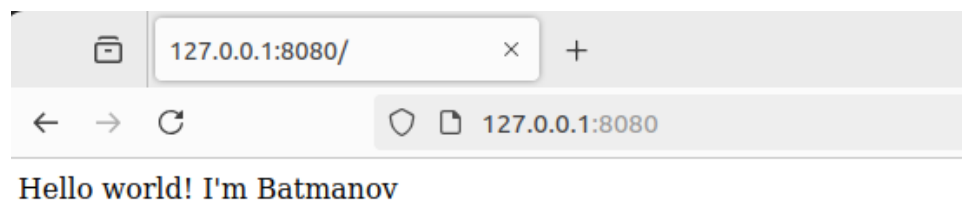


Рисунок 11 – запущенная HTML страница из контейнера

4. Запуск MariaDB

Создаем новый рабочий каталог и загружаем образ (Рисунок 12):

```

a1@a1-VirtualBox:~/nginx-project$ sudo chown -R $USER:$USER ./home/DB
a1@a1-VirtualBox:~/nginx-project$ cd /home/DB
a1@a1-VirtualBox:/home/DB$ docker pull mariadb:latest
latest: Pulling from library/mariadb
01df660f0d50: Pull complete
a3629ac5b9f4: Pull complete
5ac986756138: Pull complete
87224913ffe8: Pull complete
a57bb736d70f: Pull complete
b90af7f6e0bf: Pull complete
9a609955d447: Pull complete
096c936d41b1: Pull complete
a5cbbb0395c6: Download complete
73f5071abfa4: Download complete
Digest: sha256:f54db0cb3ccfe9431aba6d08c65a1763c499789b116b4cb651dd7fcf325965b3
Status: Downloaded newer image for mariadb:latest
docker.io/library/mariadb:latest
a1@a1-VirtualBox:/home/DB$

```

Рисунок 12 – подготовка к запуску

После этого запускаем контейнер MariaDB (Рисунок 13):

```

a1@a1-VirtualBox:/home/DB$ docker run -d --name mariadb-batmanov -p 3306:3306 -v ./home/DB:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=rootpass123 mariadb:lates
ff8cfaefcb12701474b825e889ff9ad37c43ddd2c82e92f1f6c264f7aa666274

```

Рисунок 13 – запуск контейнера

Где:

- -p 3306:3306 — порт по умолчанию для MariaDB
- -v /home/DB:/var/lib/mysql — рабочий каталог на хосте
- -e MYSQL_ROOT_PASSWORD=rootpass123 — пароль root

Проверяем, что контейнер запустился и подключаемся к базе данных (Рисунок 14):

```

a1@a1-VirtualBox:/home/DB$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
ff8cfaefcb12   mariadb:latest   "docker-entrypoint.s..." 32 seconds ago Up 32 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mariadb-batmanov
3a1f15ce9a1f   nginx-batmanov   "/docker-entrypoint..." 15 minutes ago Up 15 minutes   8080/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   web-batmanov
a1@a1-VirtualBox:/home/DB$ docker exec -it mariadb-batmanov mariadb -u root -p
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
a1@a1-VirtualBox:/home/DB$ docker exec -it mariadb-batmanov mariadb -u root -p
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
a1@a1-VirtualBox:/home/DB$ docker exec -it mariadb-batmanov mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 12.1.2-MariaDB-ubu2404 mariadb.org binary distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>

```

Рисунок 14 – подключение к базе данных

Создаем новую базу данных и проверяем, что она успешно создана (Рисунок 15):

```

MariaDB [(none)]> CREATE DATABASE batmanov;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| batmanov |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.001 sec)

MariaDB [(none)]> EXIT;
Bye
a1@a1-VirtualBox:/home/DB$

```

Рисунок 15 – создание именованной БД

В итоге видим, что БД сохранилась как отдельная директория внутри хостовой директории, которая указана как рабочий каталог для контейнера:

```

a1@a1-VirtualBox:/home/DB/home/DB$ ls
aria_log.00000001  batmanov  ib_buffer_pool  ib_logfile0  mariadb_upgrade_info  mysql  sys  undo001  undo003
aria_log_control  ddl_recovery.log  ibdata1  ibtmp1  multi-master.info  performance_schema  tc.log  undo002

```

Рисунок 16 – проверка содержимого директории

5. Создание контейнера NextCloud

Сначала база: создание разделов и загрузка образа (Рисунок 17):

```

a1@a1-VirtualBox:/home/www/NextCloud$ docker pull nextcloud:latest
latest: Pulling from library/nextcloud
Digest: sha256:0e1084cc59df77bec7d6bb29d9ac6939da8372512237a9c51f74ff0a970524f2
Status: Image is up to date for nextcloud:latest
docker.io/library/nextcloud:latest
a1@a1-VirtualBox:/home/www/NextCloud$

```

Рисунок 17 – подготовка

Далее запускаем контейнер NextCloud (Рисунок 18)

```

vaflya@WS-SSA-ubuntu:/home/www/NextCloud$ docker run -d \
--name nextcloud-app \
-p 8088:80 \
-v /home/www/NextCloud:/var/www/html \
nextcloud:latest
ad5a6af5ab9f9aa783d542fc5ae623f9d12f378ef895974bc571f5525d05326e

```

Рисунок 18 – запуск контейнера NextCloud

Где:

- -p 8088:80 — назначаем порт 8088
- -v /home/www/NextCloud:/var/www/html — рабочий каталог на хосте

Открываем localhost:8088 на котором запущен NextCloud в браузере (Рисунок 19):

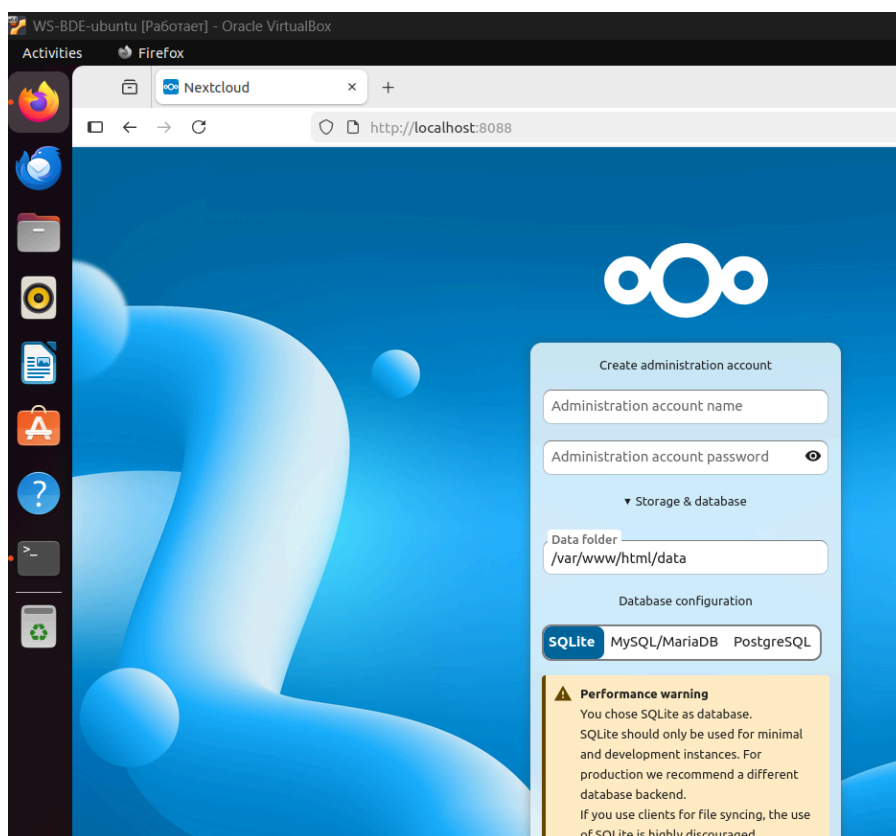


Рисунок 19 – интерфейс NextCloud в браузере

Далее создаем учетку администратора (это будет первый пользователь, то есть я сам) и нового пользователя с именем преподавателя (Рисунок 20)

Рисунок 20 – создание нового юзера

Итого имеем двух созданных пользователей (Рисунок 21)

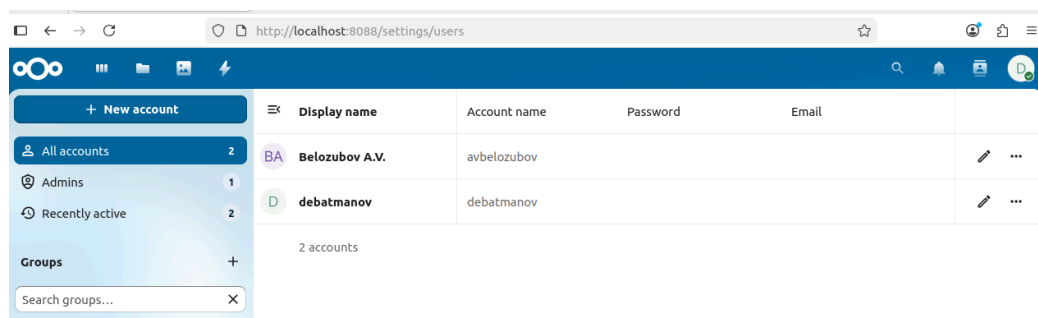


Рисунок 21 – список созданных пользователей

6. Запуск контейнера phpvirtualbox

Скачиваем образ phpvirtualbox с DockerHub (Рисунок 22)

```
a1@a1-VirtualBox:/$ docker pull jazzdd/phpvirtualbox
Using default tag: latest
latest: Pulling from jazzdd/phpvirtualbox
Digest: sha256:a881ae25a2e6349484b9c8acada28d81c07e86356ec660d6c9ba7142ef86d040
Status: Image is up to date for jazzdd/phpvirtualbox:latest
docker.io/jazzdd/phpvirtualbox:latest
a1@a1-VirtualBox:/$
```

Рисунок 22 – загрузка phpvirtualbox

Далее экспериментальным путем было определено, что после установки, чтоб все заработало, нужно запустить VirtualBox web service без аутентификации (Рисунок 23).

```
a1@a1-VirtualBox:/$ vboxwebsrv -H 0.0.0.0 --authentication null &
[1] 10005
a1@a1-VirtualBox:/$ Oracle VM VirtualBox web service Version 6.1.50_Ubuntu
(C) 2007-2024 Oracle Corporation
All rights reserved.
00:00:00.000165 main      VirtualBox web service 6.1.50_Ubuntu r161033 linux.amd6
4 (Aug  2 2024 10:31:49) release log
00:00:00.000165 main      Log opened 2024-08-15T10:42:33.733124000Z
```

Рисунок 23 – запуск web service

SOAP API сервер запустился в фоне, и вот только после этого можно запускать контейнер, указав переменные окружения (Рисунок 24):

```
docker run -d --name phpvirtualbox --net=host -e
ID_NAME=vbox -e ID_HOSTPORT=127.0.0.1:18083 -e
ID_USER=$USER -e ID_PW=rootpass123
jazzdd/phpvirtualbox
```

```
a1@a1-VirtualBox:/$ docker run -d --name phpvirtualbox --net=host -e ID_NAME=vbox
x -e ID_HOSTPORT=127.0.0.1:18083 -e ID_USER=$USER -e ID_PW=rootpass123 jazzdd/ph
pvirtualbox
73218cb54075a59be4e32fc9729f31311cc2795df6f3805980b8d2290f6c1755
a1@a1-VirtualBox:/$
```

Рисунок 24 – запуск phpvirtualbox

Где:

- -d — запуск в фоновом режиме
- --name phpvirtualbox — имя контейнера
- --net=host — использование сети хоста (доступ к VirtualBox на 127.0.0.1)
- -e ID_NAME=vbox — идентификатор сервера VirtualBox
- -e ID_HOSTPORT=127.0.0.1:18083 — адрес и порт VirtualBox SOAP API
- -e ID_USER=\$USER — имя пользователя для подключения к VirtualBox
- -e ID_PW=rootpass123 — пароль для подключения

При запуске контейнера у нас становится доступен веб-интерфейс, в котором видно запущенные виртуальные машины (Рисунок 25):

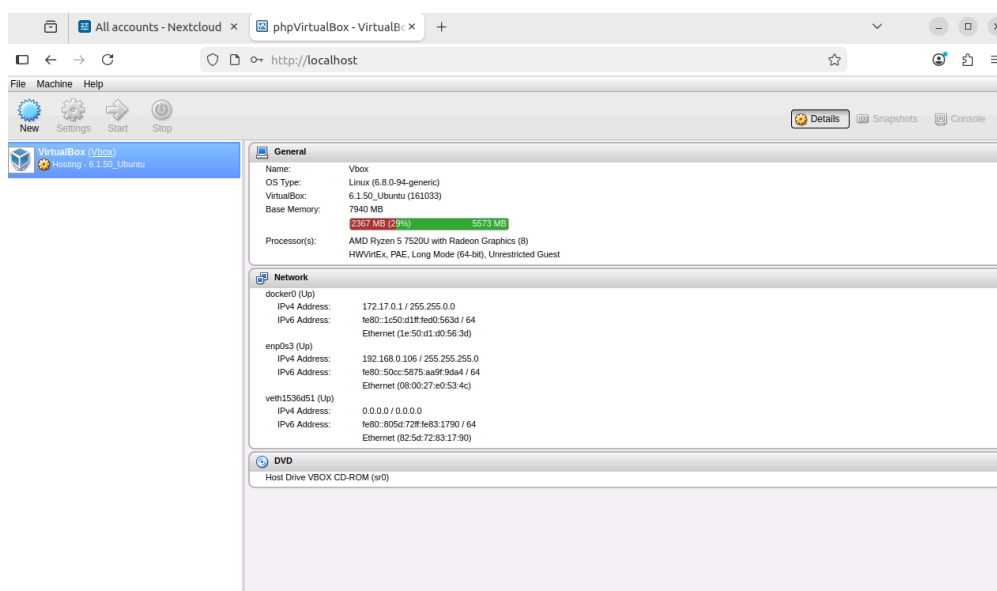


Рисунок 25 – интерфейс phpvirtualbox

По сути, он видит текущую виртуалку, с которой мы и выполняем все действия. Это происходит потому, что VirtualBox внутри Ubuntu видит информацию о хост-системе через Guest Additions.

7. Docker Compose проект – NextCloud + Nginx + MariaDB

Создаем разделы и docker-compose.yml файл, в который записываем конфигурацию (Рисунок 26):

```

version: '3'

services:
  webserver:
    image: nginx:latest
    container_name: web-batmanov
    restart: always
    ports:
      - "8080:80"
    volumes:
      - /home/www/html:/usr/share/nginx/html
    networks:
      - app-network

  database:
    image: mariadb:latest
    container_name: db-batmanov
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass123
      MYSQL_DATABASE: nextcloud_db
    volumes:
      - /home/DB:/var/lib/mysql
    networks:
      - app-network

  cloud:
    image: nextcloud:latest
    container_name: cloud-batmanov
    restart: always
    ports:
      - "2022:80"
    environment:
      MYSQL_HOST: database
      MYSQL_DATABASE: nextcloud_db
      MYSQL_USER: root
      MYSQL_PASSWORD: rootpass123
    volumes:
      - /home/www/NextCloud:/var/www/html
    depends_on:
      - database
      - webserver
    networks:

```

```

networks:
  app-network:
    driver: bridge

```

Рисунок 26 – содержимое docker-compose

Запускаем контейнеры и проверяем, что все успешно запущены (Рисунок 27):

```

a1@a1-VirtualBox:/nextcloud-compose$ docker-compose up
db-batmanov is up-to-date
Creating web-batmanov ... done
Creating cloud-batmanov ... done
Attaching to db-batmanov, web-batmanov, cloud-batmanov
web-batmanov | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
web-batmanov | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-batmanov | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web-batmanov | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
web-batmanov | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf

a1@a1-VirtualBox:/nextcloud-compose$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
689e66f9db39   nextcloud:late /entrypoint.sh apac...   22 seconds ago Up 21 seconds 0.0.0.0:2022->80/tcp, [::]:2022->80/tcp  cloud-batmanov
8b2528041623   nginx:latest   "/docker-entrypoint..." 23 seconds ago Up 21 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  web-batmanov
693563ebf940   mariadb:latest "docker-entrypoint.s..." About a minute ago   Up About a minute  3306/tcp                               db-batmanov
a1@a1-VirtualBox:/nextcloud-compose$

```

Рисунок 27 – запуск контейнеров через compose

Проверяем и видим, что на порту 2022 успешно запустился NextCloud:

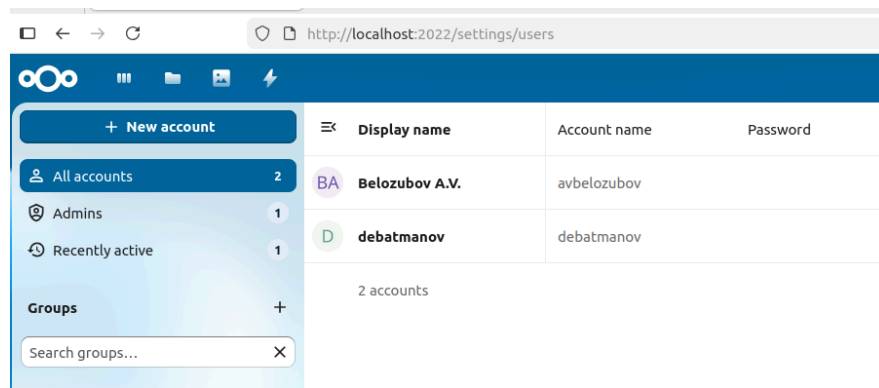


Рисунок 28 – запущенный NextCloud

8. Регистрация и загрузка на DockerHub

Произвел регистрацию на DockerHub (Рисунок 29):

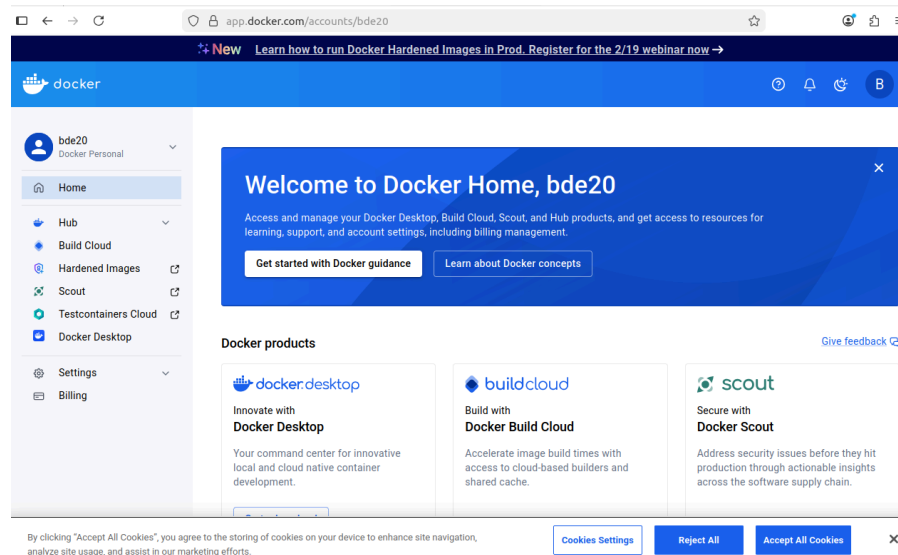


Рисунок 29 – интерфейс DockerHub после регистрации

После чего залогинился в аккаунт Докера в терминале (Рисунок 30):

```
a1@a1-VirtualBox:/nextcloud-compose$ docker login -u bde20
Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/home/a1/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
a1@a1-VirtualBox:/nextcloud-compose$
```

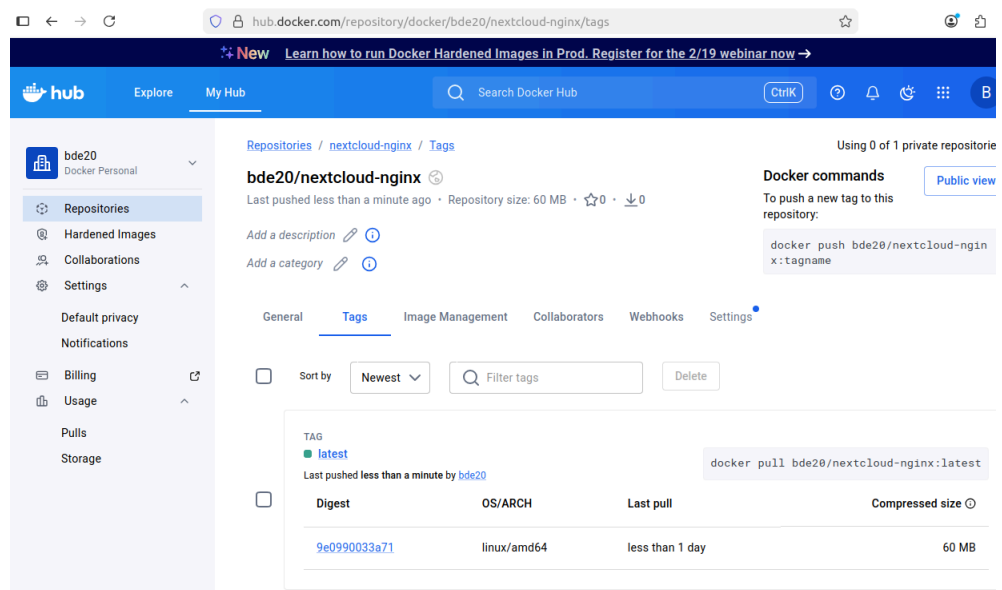
Рисунок 30 – успешный логин в аккаунт Докера

Далее собираем кастомный образ с тегом сервиса Nginx с HTML страницей из моего проекта compose и пушим его на DockerHub (Рисунок 31):

```
ai@a1-VirtualBox:/nextcloud-compose$ sudo nano nginx.Dockerfile
ai@a1-VirtualBox:/nextcloud-compose$ docker build -f nginx.Dockerfile -t bde20/nextcloud-nginx:latest .
[+] Building 1.0s (7/7) FINISHED
=> [internal] load build definition from nginx.Dockerfile
=> => transferring dockerfile: 112B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> => transferring context: 28
=> [internal] load build context
=> => transferring context: 109B
=> [1/2] FROM docker.io/library/nginx:latest@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
=> => resolve docker.io/library/nginx:latest@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
=> [2/2] COPY index.html /usr/share/nginx/html/
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:9e0990033a71a4a1e91ccf10c9933904cdc0f77a0100a4b6ede535ee9c44ac92
=> => exporting config sha256:6f5b27210cc92679ff04bbf6c1a93283169122150fc25bacf6975f3ee8bdc107
=> => exporting attestation manifest sha256:1e1c67f5546b7e742b3b36ebe7be02431d52641630e3118ab76aa5e09ef1c6f2
=> => exporting manifest list sha256:dd05117cb113e9b26a6a296e8a6a3cfacd957c1e9f2912182034b57ec30ed28c
=> => naming to docker.io/bde20/nextcloud-nginx:latest
=> => unpacking to docker.io/bde20/nextcloud-nginx:latest
ai@a1-VirtualBox:/nextcloud-compose$ docker push bde20/nextcloud-nginx:latest
The push refers to repository [docker.io/bde20/nextcloud-nginx]
46bf3a120c8e: Mounted from library/nginx
4f4efe02d542: Mounted from library/nginx
7b6cb8ccac7b: Mounted from library/nginx
47cd406a84ef: Mounted from library/nginx
15dd9170335a: Pushed
0c8d55a45c0d: Mounted from library/nextcloud
f73400a233fd: Mounted from library/nginx
bae5a1799a80: Mounted from library/nginx
2576eba76711: Pushed
latest: digest: sha256:dd05117cb113e9b26a6a296e8a6a3cfacd957c1e9f2912182034b57ec30ed28c size: 856
ai@a1-VirtualBox:/nextcloud-compose$ sudo nano docker-compose.yml
```

Рисунок 31 – сборка образа с тегом и пуш

Как видим, образ успешно запустился и доступен в моем публичном репозитории (Рисунок 32)



The screenshot shows the Docker Hub interface for the repository `bde20/nextcloud-nginx`. The page includes a sidebar with navigation options like `Repositories`, `Hardened Images`, `Collaborations`, `Settings`, `Default privacy`, `Notifications`, `Billing`, `Usage`, `Pulls`, and `Storage`. The main content area displays the repository details, including the tag `latest` and the Docker commands to push and pull the image. The `Tags` tab is selected, showing a list of tags with columns for `Tag`, `Digest`, `OS/ARCH`, `Last pull`, and `Compressed size`. The `latest` tag is highlighted, showing a digest of `9e0990033a71` and a size of `60 MB`.

Рисунок 32 – опубликованный образ

9. Запуск готового приложения с DockerHub

Рабочего образа 2048 не нашел, а установка клиента Minecraft – довольно трудоемкая задача, поэтому нашел и установил образ Тетриса (Рисунок 33):

```

a1@a1-VirtualBox:/nextcloud-compose$ docker pull bsord/tetris
Using default tag: latest
latest: Pulling from bsord/tetris
802dad68ade4: Pull complete
352e5a6cac26: Pull complete
be0c016df0be: Pull complete
6c0ee9353e13: Pull complete
9eaf108767c7: Pull complete
dca7733b187e: Pull complete
ae13dd578326: Pull complete
Digest: sha256:2ffffd2f8834dd13734654eeec142aebd95e6880e52d692a9e59fd9aa2f9634
Status: Downloaded newer image for bsord/tetris:latest
docker.io/bsord/tetris:latest
a1@a1-VirtualBox:/nextcloud-compose$ docker run -d -p 8082:80 --name tetris bsord/tetris
e89ca04b15b2d8a1da1be470a42b63328be2381039c07e76dd086f81cfbdefc8
a1@a1-VirtualBox:/nextcloud-compose$

```

Рисунок 33 – установка и запуск контейнера с Тетрисом

После этого открыл localhost:8082 в браузере и убедился, что все работает (Рисунок 34):

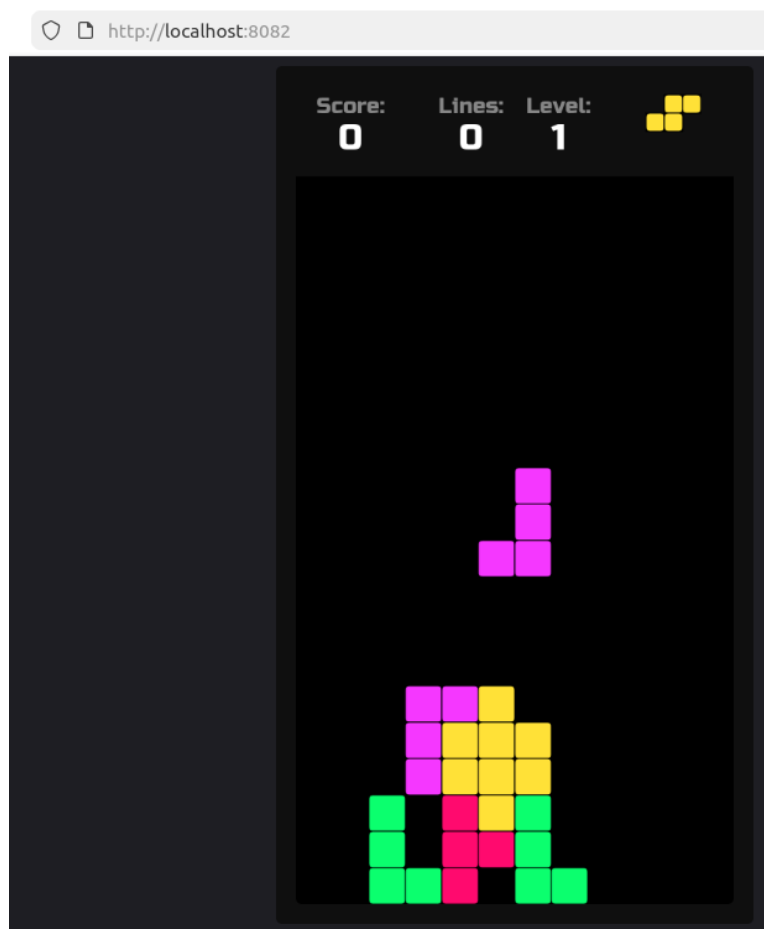


Рисунок 34 – запущенный в браузере Тетрис

10. Основные команды Docker

Проверка версии

`docker --version`

Управление образами

`docker pull <image>` # Скачать образ
`docker images` # Список образов
`docker build -t <name> .` # Собрать образ из Dockerfile
`docker rmi <image>` # Удалить образ
`docker image prune -a` # Удалить неиспользуемые образы

Управление контейнерами

`docker run <options> <image>` Запустить контейнер
`docker ps` Список запущенных контейнеров
`docker ps -a` Список всех контейнеров
`docker stop <container>` Остановить контейнер
`docker start <container>` Запустить контейнер
`docker restart <container>` Перезапустить контейнер
`docker rm <container>` Удалить контейнер
`docker logs <container>` Посмотреть логи
`docker exec -it <container> bash` Зайти внутрь контейнера

Docker Compose

`docker-compose up -d` Запустить проект
`docker-compose down` Остановить проект
`docker-compose ps` Статус контейнеров

DockerHub

`docker login` Войти в DockerHub
`docker tag <image> <user>/<name>` Создать тег
`docker push <user>/<name>` Загрузить на DockerHub

Очистка

`docker system prune -a --volumes` Полная очистка

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было развернуто окружение для работы с платформой контейнеризации Docker в операционной системе Ubuntu, функционирующей на базе гипервизора Oracle VirtualBox.

Освоены базовые команды Docker для управления контейнерами и образами. Создан первый пользовательский образ с использованием команды `docker build`, демонстрирующий вывод приветственного сообщения с указанием фамилии.

Разработан и развернут веб-сервер на базе Nginx с использованием Dockerfile. Создана персональная HTML-страница с информацией о студенте, настроен проброс порта 8080 и установлен рабочий каталог `/home/www/html` на хосте для размещения веб-контента. Контейнер успешно запущен и протестирован через браузер.

Развернут контейнер с СУБД MariaDB с конфигурацией рабочего каталога `/home/DB` и портом по умолчанию 3306. Выполнено подключение к базе данных через терминальный клиент внутри контейнера, создана база данных с именем соответствующим фамилии студента.

Настроена платформа облачного хранилища NextCloud на порту 8088 с рабочим каталогом `/home/www/NextCloud`. Выполнена первоначальная конфигурация системы с созданием учетной записи администратора и регистрацией дополнительного пользователя согласно требованиям задания.

Развернут веб-интерфейс phpVirtualBox для удаленного управления виртуальными машинами VirtualBox. Настроено подключение к VirtualBox web service через SOAP API с использованием переменных окружения для аутентификации.

Создан многокомпонентный проект с использованием Docker Compose, объединяющий три взаимосвязанных сервиса: веб-сервер Nginx, систему управления базами данных MariaDB и платформу NextCloud. Настроена оркестрация контейнеров с автоматическим управлением зависимостями через `docker-compose.yml`. Проект развернут на порту 2022 с использованием виртуальной сети bridge для межконтейнерного взаимодействия.

Выполнена регистрация на платформе DockerHub. Создан пользовательский образ на основе Nginx с включением персональной HTML-страницы, присвоен тег в формате `username/image:tag` и опубликован в публичном реестре Docker Hub с использованием команды `docker push`.

Продemonстрирована работа с готовыми образами из публичного реестра DockerHub путем развертывания веб-приложения игры Tetris, что подтверждает возможность быстрого развертывания готовых решений через платформу контейнеризации.

Систематизированы и задокументированы основные команды Docker, использованные в процессе выполнения работы, включая управление образами, контейнерами, работу с Docker Compose и взаимодействие с облачным реестром DockerHub.

В результате работы получены практические навыки работы с технологиями контейнеризации, освоены принципы создания Dockerfile, управления жизненным циклом контейнеров, оркестрации многокомпонентных приложений через Docker Compose и публикации образов в облачные реестры.